

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5

8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I MONIKA.B (192571002) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: MONIKA.B

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided

	justification			
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Online Shopping Platform – Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Analyze the case: an online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Step 1 — Understanding of the Case

Candidate key: (OrderID, ProdID)

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
OrderID → CustID	Each order is placed by exactly one customer
CustID → CustName	Each customer ID maps to one customer name
ProdID → ProdName, Price	Each product has one name and one price
(OrderID, ProdID) → Qty	Quantity depends on the order-product combination

Step 3 — Analysis (Anomalies in the Unnormalized Table)

Insertion: cannot add a new product without an order. Update: CustName/ProdName repeated in every row, so a name change requires many updates. Deletion: deleting the last order of a customer/product loses that customer's/product's details.

Step 4 — Solution Design (Normalization Steps)

- 1NF: All attributes already hold atomic (single) values — the table is in 1NF.
- 2NF: The key is composite (OrderID, ProdID). OrderID → CustID, CustName and ProdID → ProdName, Price are partial dependencies (they depend on only part of the key). Remove them into separate tables.
- 3NF: In the Orders table, OrderID → CustID → CustName is a transitive dependency. Remove it by splitting Customers out.

Final Normalized Schema

- Customers(CustID, CustName) — PK: CustID
- Products(ProdID, ProdName, Price) — PK: ProdID
- Orders(OrderID, CustID) — PK: OrderID, FK: CustID → Customers
- OrderDetails(OrderID, ProdID, Qty) — PK: (OrderID, ProdID), FK: OrderID → Orders, ProdID → Products

SQL Implementation :

```
Run SQLite
9
10 INSERT INTO Customers VALUES (1,'Ravi'),(2,'Anu');
11 INSERT INTO Products VALUES (101,'Pen',10),(102,'Bag',500);
12 INSERT INTO Orders VALUES (5001,1),(5002,2);
13 INSERT INTO OrderDetails VALUES (5001,101,5),(5001,102,1),(5002,101,2);
14
15 SELECT o.OrderID, c.CustName, p.ProdName, od.Qty, p.Price
16 FROM OrderDetails od
17 JOIN Orders o ON od.OrderID=o.OrderID
18 JOIN Customers c ON o.CustID=c.CustID
19 JOIN Products p ON od.ProdID=p.ProdID;
20
21
```

OrderID	CustName	ProdName	Qty	Price
5001	Ravi	Pen	5	10
5001	Ravi	Bag	1	500
5002	Anu	Pen	2	10

Q2. Hospital – Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Step 1 — Understanding of the Case

Candidate key: PID

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
PID → PName, DocID, WardNo	Each patient has one name, one attending doctor, one ward
DocID → DocName, Specialization	Each doctor has one name and one specialization
WardNo → WardName	Each ward number has one ward name

Step 3 — Analysis (Anomalies in the Unnormalized Table)

Update: a doctor's specialization is repeated for every patient under that doctor — changing it means updating many rows.
Insertion: a new doctor cannot be added until assigned a patient. Deletion: discharging the only patient in a ward loses that ward's name.

Step 4 — Solution Design (Normalization Steps)

- 1NF: satisfied — atomic attributes.

- 2NF/3NF: PID is a single-attribute key, so no partial dependency exists; but DocID → DocName, Specialization and WardNo → WardName are transitive on PID, violating 3NF.
- BCNF check: for every FD, the left side must be a candidate key. DocID → DocName/Specialization and WardNo → WardName violate this (DocID, WardNo are not keys of the whole relation) — decompose.

Final Normalized Schema

- Doctors(DocID, DocName, Specialization) — PK: DocID
- Wards(WardNo, WardName) — PK: WardNo
- Patients(PID, PName, DocID, WardNo) — PK: PID, FK: DocID → Doctors, WardNo → Wards

SQL Implementation :

```

Run SQLite
1 CREATE TABLE Doctors (DocID INTEGER PRIMARY KEY, DocName TEXT, Specialization TEXT);
2 CREATE TABLE Wards (WardNo INTEGER PRIMARY KEY, WardName TEXT);
3 CREATE TABLE Patients (PID INTEGER PRIMARY KEY, PName TEXT, DocID INTEGER, WardNo INTEGER,
4   FOREIGN KEY(DocID) REFERENCES Doctors(DocID),
5   FOREIGN KEY(WardNo) REFERENCES Wards(WardNo));
6
7 INSERT INTO Doctors VALUES (1,'Dr. Kumar','Cardiology'),(2,'Dr. Iyer','Orthopedics');
8 INSERT INTO Wards VALUES (10,'General'),(11,'ICU');
9 INSERT INTO Patients VALUES (201,'Meena',1,10),(202,'Suresh',2,11);
10
11 SELECT p.PName, d.DocName, d.Specialization, w.WardName
12 FROM Patients p
13 JOIN Doctors d ON p.DocID=d.DocID

```

PName	DocName	Specialization	WardName
Meena	Dr. Kumar	Cardiology	General
Suresh	Dr. Iyer	Orthopedics	ICU

Q3. University – Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

Step 1 — Understanding of the Case

Candidate key: (SID, CourseID)

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
$SID \rightarrow SName$	Each student has one name
$CourseID \rightarrow CourseName, Faculty$	Each course has one name and one faculty in charge
$(SID, CourseID) \rightarrow Grade$	Grade depends on the student-course combination

Step 3 — Analysis (Anomalies in the Unnormalized Table)

SName and CourseName/Faculty repeat across many rows; a faculty change for a course requires updating every enrolled student's row.

Step 4 — Solution Design (Normalization Steps)

- 1NF: satisfied.
- 2NF: $SID \rightarrow SName$ and $CourseID \rightarrow CourseName, Faculty$ are partial dependencies on the composite key (SID, CourseID). Remove them into their own tables, leaving only the full-key-dependent attribute (Grade) in Enrollment.

Final Normalized Schema

- Students(SID, SName) — PK: SID
- Courses(CourseID, CourseName, Faculty) — PK: CourseID
- Enrollment(SID, CourseID, Grade) — PK: (SID, CourseID), FK: $SID \rightarrow Students$, $CourseID \rightarrow Courses$

SQL Implementation :

Run

SQLite

abc

```
1 CREATE TABLE Students (SID INTEGER PRIMARY KEY, SName TEXT);
2 CREATE TABLE Courses (CourseID INTEGER PRIMARY KEY, CourseName TEXT, Faculty TEXT);
3 CREATE TABLE Enrollment (SID INTEGER, CourseID INTEGER, Grade TEXT,
4   PRIMARY KEY(SID,CourseID),
5   FOREIGN KEY(SID) REFERENCES Students(SID),
6   FOREIGN KEY(CourseID) REFERENCES Courses(CourseID));
7
8 INSERT INTO Students VALUES (1,'Arun'),(2,'Divya');
9 INSERT INTO Courses VALUES (301,'DBMS','Dr. Rao'),(302,'OS','Dr. Nair');
10 INSERT INTO Enrollment VALUES (1,301,'A'),(1,302,'B'),(2,301,'A+');
11
12 SELECT s.SName, c.CourseName, c.Faculty, e.Grade
13 FROM Enrollment e
```

SName	CourseName	Faculty	Grade
Arun	DBMS	Dr. Rao	A
Arun	OS	Dr. Nair	B
Divya	DBMS	Dr. Rao	A+



Q4. Airline Booking – Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

Step 1 — Understanding of the Case

Candidate key: (FlightNo, Date, PassID)

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
FlightNo → DeptCity, ArrCity	Each flight number has a fixed route
PassID → PassName	Each passenger has one name
(FlightNo, Date, PassID) → Seat	Seat is assigned per flight instance per passenger

Step 3 — Analysis (Anomalies in the Unnormalized Table)

Route (DeptCity/ArrCity) repeats for every booking of that flight; PassName repeats for every flight a passenger takes.

Step 4 — Solution Design (Normalization Steps)

- 1NF: satisfied.
- 2NF: FlightNo → DeptCity, ArrCity and PassID → PassName are partial dependencies on the key (FlightNo, Date, PassID) — remove into separate tables.
- 3NF/BCNF: In the decomposed tables every remaining FD's left side is a candidate key (FlightNo for Flights, PassID for Passengers, the full composite key for Bookings) — the schema is already in BCNF.

Final Normalized Schema

- Flights(FlightNo, DeptCity, ArrCity) — PK: FlightNo
- Passengers(PassID, PassName) — PK: PassID
- Bookings(FlightNo, TravelDate, PassID, Seat) — PK: (FlightNo, TravelDate, PassID), FK: FlightNo → Flights, PassID → Passengers

SQL Implementation :

Run

SQLite

abc

```
1 CREATE TABLE Flights (FlightNo TEXT PRIMARY KEY, DeptCity TEXT, ArrCity TEXT);
2 CREATE TABLE Passengers (PassID INTEGER PRIMARY KEY, PassName TEXT);
3 CREATE TABLE Bookings (FlightNo TEXT, TravelDate TEXT, PassID INTEGER, Seat TEXT,
4   PRIMARY KEY(FlightNo, TravelDate, PassID),
5   FOREIGN KEY(FlightNo) REFERENCES Flights(FlightNo),
6   FOREIGN KEY(PassID) REFERENCES Passengers(PassID));
7
8 INSERT INTO Flights VALUES ('AI202','Chennai','Delhi');
9 INSERT INTO Passengers VALUES (1,'Karthik'),(2,'Priya');
10 INSERT INTO Bookings VALUES ('AI202','2026-08-20',1,'12A'),('AI202','2026-08-20',2,'12B');
11
12 SELECT b.FlightNo, f.DeptCity, f.ArrCity, p.PassName, b.Seat, b.TravelDate
13 FROM Bookings b
```

FlightNo	DeptCity	ArrCity	PassName	Seat	TravelDate
AI202	Chennai	Delhi	Karthik	12A	2026-08-20
AI202	Chennai	Delhi	Priya	12B	2026-08-20

Q5. HR System (Multinational Company) – Employee/Department schema

Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

Step 1 — Understanding of the Case

Assumed unnormalized relation: HR(EmpID, EmpName, DeptID, DeptName, DeptLocation, ManagerID, Salary). Candidate key: EmpID

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
EmpID → EmpName, DeptID, ManagerID, Salary	Each employee has one name, department, manager, salary
DeptID → DeptName, DeptLocation	Each department has one name and one location

Step 3 — Analysis (Anomalies in the Unnormalized Table)

Transitive dependency EmpID → DeptID → DeptName/DeptLocation means department details repeat for every employee in that department — update anomaly if a department relocates; deletion anomaly if the last employee in a department is removed.

Step 4 — Solution Design (Normalization Steps)

- 1NF: satisfied.
- 2NF: EmpID is a single-attribute key, so no partial dependency exists.
- 3NF: EmpID → DeptID → DeptName, DeptLocation is transitive (DeptName/DeptLocation depend on EmpID only through DeptID). Remove the transitive dependency by splitting out Departments.

Final Normalized Schema

- Departments(DeptID, DeptName, DeptLocation) — PK: DeptID
- Employees(EmpID, EmpName, DeptID, ManagerID, Salary) — PK: EmpID, FK: DeptID → Departments

SQL Implementation :

Run

SQLite

abc

```
1 CREATE TABLE Departments (DeptID INTEGER PRIMARY KEY, DeptName TEXT, DeptLocation TEXT);
2 CREATE TABLE Employees (EmpID INTEGER PRIMARY KEY, EmpName TEXT, DeptID INTEGER,
3   ManagerID INTEGER, Salary REAL,
4   FOREIGN KEY(DeptID) REFERENCES Departments(DeptID));
5
6 INSERT INTO Departments VALUES (1,'IT','Chennai'),(2,'HR','Bangalore');
7 INSERT INTO Employees VALUES (1,'Vikram',1,NULL,90000),(2,'Sneha',1,1,60000),(3,'Ajay',2,NULL,55000);
8
9 SELECT e.EmpName, d.DeptName, d.DeptLocation, e.Salary
10 FROM Employees e
11 JOIN Departments d ON e.DeptID=d.DeptID;
12
```

EmpName	DeptName	DeptLocation	Salary
Vikram	IT	Chennai	90000
Sneha	IT	Chennai	60000
Ajay	HR	Bangalore	55000

Q6. Library – Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Step 1 — Understanding of the Case

The relation is already in BCNF on (MemberID, BookID, Author, Genre, BorrowDate) once trivial FDs are removed, but it hides two independent multivalued facts about a book.

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
BookID ->> Author (MVD)	A book can have multiple authors, independent of its genres
BookID ->> Genre (MVD)	A book can belong to multiple genres, independent of its authors
MemberID → MemberName	Each member has one name
BookID → Title	Each book has one title

Step 3 — Analysis (Anomalies in the Unnormalized Table)

Storing Author and Genre in the same row forces every Author to be paired with every Genre for that book (a Cartesian product inside one row set), creating redundant rows and update anomalies whenever an author or genre list changes.

Step 4 — Solution Design (Normalization Steps)

- 1NF/2NF/3NF/BCNF: the relation has no partial or transitive FD anomaly left, but it violates 4NF because it contains two independent multivalued dependencies on BookID (Author and Genre).
- 4NF: split the two independent multivalued facts into separate tables, each holding BookID with just one of the multivalued attributes.

Final Normalized Schema

- Members(MemberID, MemberName) — PK: MemberID
- Books(BookID, Title) — PK: BookID
- BookAuthors(BookID, Author) — PK: (BookID, Author), FK: BookID → Books
- BookGenres(BookID, Genre) — PK: (BookID, Genre), FK: BookID → Books
- Borrow(MemberID, BookID, BorrowDate) — PK: (MemberID, BookID, BorrowDate), FK: MemberID → Members, BookID → Books

SQL Implementation :

Run

SQLite

```
1 CREATE TABLE Members (MemberID INTEGER PRIMARY KEY, MemberName TEXT);
2 CREATE TABLE Books (BookID INTEGER PRIMARY KEY, Title TEXT);
3 CREATE TABLE BookAuthors (BookID INTEGER, Author TEXT, PRIMARY KEY(BookID, Author));
4 CREATE TABLE BookGenres (BookID INTEGER, Genre TEXT, PRIMARY KEY(BookID, Genre));
5 CREATE TABLE Borrow (MemberID INTEGER, BookID INTEGER, BorrowDate TEXT,
6   PRIMARY KEY(MemberID, BookID, BorrowDate));
7
8 INSERT INTO Members VALUES (1,'Harini');
9 INSERT INTO Books VALUES (10,'Database Systems');
10 INSERT INTO BookAuthors VALUES (10,'Elmasri'),(10,'Navathe');
11 INSERT INTO BookGenres VALUES (10,'Technical'),(10,'Reference');
12 INSERT INTO Borrow VALUES (1,10,'2026-08-01');
13
```

MemberName	Title	Author	Genre	BorrowDate
Harini	Database Systems	Elmasri	Reference	2026-08-01
Harini	Database Systems	Elmasri	Technical	2026-08-01
Harini	Database Systems	Navathe	Reference	2026-08-01
Harini	Database Systems	Navathe	Technical	2026-08-01

Q7. Telecom Billing – Candidate keys, primary key, FDs before normalization

Analyze a telecom billing schema and identify all candidate keys, primary key, and functional dependencies before normalization.

Step 1 — Understanding of the Case

Assumed unnormalized relation: Billing(PhoneNo, CustID, CustName, PlanID, PlanName, BillMonth, Amount).

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
PhoneNo $\rightarrow$ CustID, PlanID	Each phone number belongs to one customer on one plan
CustID $\rightarrow$ CustName	Each customer ID maps to one name
PlanID $\rightarrow$ PlanName	Each plan ID maps to one plan name
(PhoneNo, BillMonth) $\rightarrow$ Amount	The bill amount depends on the phone number and the billing month together

Step 3 — Analysis (Anomalies in the Unnormalized Table)

CustName and PlanName repeat in every monthly bill row for the same phone — a plan rename or customer name change must be repeated across many rows.

Step 4 — Solution Design (Normalization Steps)

- Candidate keys: only (PhoneNo, BillMonth) functionally determines every other attribute (via PhoneNo $\rightarrow$ CustID, PlanID $\rightarrow$ CustName, PlanName, and directly $\rightarrow$ Amount), so it is the sole candidate key.
- Primary key selected: (PhoneNo, BillMonth).
- Normalization to 3NF/BCNF: PhoneNo $\rightarrow$ CustID, PlanID is a partial dependency on the key, and CustID $\rightarrow$ CustName, PlanID $\rightarrow$ PlanName are transitive — decompose into Customers, Plans, Phones, and Billing.

Final Normalized Schema

- Customers(CustID, CustName) — PK: CustID
- Plans(PlanID, PlanName) — PK: PlanID
- Phones(PhoneNo, CustID, PlanID) — PK: PhoneNo, FK: CustID $\rightarrow$ Customers, PlanID $\rightarrow$ Plans
- Billing(PhoneNo, BillMonth, Amount) — PK: (PhoneNo, BillMonth), FK: PhoneNo $\rightarrow$ Phones

SQL Implementation :

```
Run SQLite
8 FOREIGN KEY(PhoneNo) REFERENCES Phones(PhoneNo));
9
10 INSERT INTO Customers7 VALUES (1,'Naveen');
11 INSERT INTO Plans VALUES (1,'Unlimited 199');
12 INSERT INTO Phones VALUES ('9840000001',1,1);
13 INSERT INTO Billing VALUES ('9840000001','2026-07',199),('9840000001','2026-08',199);
14
15 SELECT ph.PhoneNo, c.CustName, pl.PlanName, bi.BillMonth, bi.Amount
16 FROM Billing bi
17 JOIN Phones ph ON bi.PhoneNo=ph.PhoneNo
18 JOIN Customers7 c ON ph.CustID=c.CustID
19 JOIN Plans pl ON ph.PlanID=pl.PlanID;
20
```

PhoneNo	CustName	PlanName	BillMonth	Amount
9840000001	Naveen	Unlimited 199	2026-07	199
9840000001	Naveen	Unlimited 199	2026-08	199

Q8. Retail Inventory – Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Step 1 — Understanding of the Case

Candidate key: PID

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
PID → PName, SupplierID, Category, Price, Warehouse	Each product has one name, supplier, category, price, warehouse
SupplierID → SupplierName	Each supplier ID maps to one supplier name

Step 3 — Analysis (Anomalies in the Unnormalized Table)

SupplierName repeats for every product from the same supplier — a supplier name correction requires updating many rows; a new supplier cannot be recorded until it supplies a product.

Step 4 — Solution Design (Normalization Steps)

- 1NF/2NF: satisfied — PID is a single-attribute key, so no partial dependency.
- 3NF: PID → SupplierID → SupplierName is transitive — remove by splitting out Suppliers.
- BCNF check: in both resulting tables, every FD's left side (PID, SupplierID) is a candidate key of its own table — normalization complete.

Final Normalized Schema

- Suppliers(SupplierID, SupplierName) — PK: SupplierID
- Products(PID, PName, SupplierID, Category, Price, Warehouse) — PK: PID, FK: SupplierID → Suppliers

SQL Implementation :

```
Run SQLite
1 CREATE TABLE Suppliers (SupplierID INTEGER PRIMARY KEY, SupplierName TEXT);
2 CREATE TABLE Products8 (PID INTEGER PRIMARY KEY, PName TEXT, SupplierID INTEGER,
3 Category TEXT, Price REAL, Warehouse TEXT,
4 FOREIGN KEY(SupplierID) REFERENCES Suppliers(SupplierID));
5
6 INSERT INTO Suppliers VALUES (1,'ABC Traders'),(2,'XYZ Corp');
7 INSERT INTO Products8 VALUES (1,'Rice Bag',1,'Grocery',1200,'WH1'),(2,'Notebook',2,'Stationery',40,'WH2');
8
9 SELECT p.PName, s.SupplierName, p.Category, p.Price, p.Warehouse
10 FROM Products8 p
11 JOIN Suppliers s ON p.SupplierID=s.SupplierID;
12
13
```

PName	SupplierName	Category	Price	Warehouse
Rice Bag	ABC Traders	Grocery	1200	WH1
Notebook	XYZ Corp	Stationery	40	WH2

Q9. School ERP – Poorly normalized schema, redesign to BCNF

Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Step 1 — Understanding of the Case

Assumed unnormalized relation: SchoolERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, Subject, Marks, ParentName, ParentContact).

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
StudentID → StudentName, ClassID, ParentName, ParentContact	Each student has one name, class, and parent contact
ClassID → ClassName, TeacherID	Each class has one name and one class teacher
TeacherID → TeacherName	Each teacher ID maps to one name
(StudentID, Subject) → Marks	Marks depend on the student-subject combination

Step 3 — Analysis (Anomalies in the Unnormalized Table)

ClassName/TeacherID repeat for every student in the class; TeacherName repeats for every class the teacher handles — a teacher reassignment or class rename requires many updates, and a class with no students yet cannot be recorded.

Step 4 — Solution Design (Normalization Steps)

- 1NF: satisfied.
- 2NF: (StudentID, Subject) is the natural key for the Marks fact, but StudentID → StudentName, ClassID, ParentName, ParentContact is a partial dependency — remove into a Students table.
- 3NF: ClassID → ClassName, TeacherID and TeacherID → TeacherName are transitive on StudentID — remove into Classes and Teachers tables.
- BCNF check: every remaining FD's left side is a key of its table — the design is in BCNF.

Final Normalized Schema

- Teachers(TeacherID, TeacherName) — PK: TeacherID
- Classes(ClassID, ClassName, TeacherID) — PK: ClassID, FK: TeacherID → Teachers
- Students(StudentID, StudentName, ClassID, ParentName, ParentContact) — PK: StudentID, FK: ClassID → Classes
- Marks(StudentID, Subject, Marks) — PK: (StudentID, Subject), FK: StudentID → Students

SQL Implementation :

```
Run SQLite
9 FOREIGN KEY(StudentID) REFERENCES Students9(StudentID));
10
11 INSERT INTO Teachers VALUES (1,'Mrs. Latha');
12 INSERT INTO Classes VALUES (1,'10-A',1);
13 INSERT INTO Students9 VALUES (1,'Nithya',1,'Mr. Ganesh','9840011122');
14 INSERT INTO Marks VALUES (1,'Maths',92),(1,'Science',88);
15
16 SELECT s.StudentName, c.ClassName, t.TeacherName, m.Subject, m.Marks
17 FROM Students9 s
18 JOIN Classes c ON s.ClassID=c.ClassID
19 JOIN Teachers t ON c.TeacherID=t.TeacherID
20 JOIN Marks m ON s.StudentID=m.StudentID;
21
```

StudentName	ClassName	TeacherName	Subject	Marks
Nithya	10-A	Mrs. Latha	Maths	92
Nithya	10-A	Mrs. Latha	Science	88

Q10. Movie Streaming Platform – Join dependency, decompose to 5NF

Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Step 1 — Understanding of the Case

Assumed ternary relation: Streaming(MovieID, ActorID, PlatformID), recording which actor of which movie appears on which platform, where the three pairwise relationships hold independently.

Step 2 — Application of Concepts (Functional Dependencies)

Functional Dependency	Meaning
-----------------------	---------

JD: $*(\text{MovieID}, \text{ActorID}), (\text{MovieID}, \text{PlatformID}), (\text{ActorID}, \text{PlatformID})$	The ternary fact is fully reconstructible from its three pairwise projections — no FD or MVD alone captures the constraint, only a join dependency does
---	---

Step 3 — Analysis (Anomalies in the Unnormalized Table)

Storing the ternary relation directly forces redundant rows whenever the same movie-actor, movie-platform, and actor-platform pairings recur across multiple entries, and risks losing information if one pairing is deleted without the others.

Step 4 — Solution Design (Normalization Steps)

- 1NF–4NF: the relation has no partial/transitive FD and no non-trivial MVD, so it already satisfies 1NF–4NF.
- 5NF: the relation still satisfies a join dependency that is not implied by any candidate key, so it is decomposed into its three binary projections.
- Lossless-join verification: joining $\text{MovieActor} \bowtie \text{MoviePlatform} \bowtie \text{ActorPlatform}$ on the shared attributes must reproduce exactly the original tuples with no spurious rows — verified below by re-running the natural join over sample data and comparing the result to the original ternary fact.

Final Normalized Schema

- $\text{MovieActor}(\text{MovieID}, \text{ActorID})$ — PK: $(\text{MovieID}, \text{ActorID})$
- $\text{MoviePlatform}(\text{MovieID}, \text{PlatformID})$ — PK: $(\text{MovieID}, \text{PlatformID})$
- $\text{ActorPlatform}(\text{ActorID}, \text{PlatformID})$ — PK: $(\text{ActorID}, \text{PlatformID})$

SQL Implementation :

```

Run SQLite
1 CREATE TABLE MovieActor (MovieID INTEGER, ActorID INTEGER, PRIMARY KEY(MovieID, ActorID));
2 CREATE TABLE MoviePlatform (MovieID INTEGER, PlatformID INTEGER, PRIMARY KEY(MovieID, PlatformID));
3 CREATE TABLE ActorPlatform (ActorID INTEGER, PlatformID INTEGER, PRIMARY KEY(ActorID, PlatformID));
4
5 INSERT INTO MovieActor VALUES (1,1),(1,2);
6 INSERT INTO MoviePlatform VALUES (1,10),(1,20);
7 INSERT INTO ActorPlatform VALUES (1,10),(1,20),(2,10),(2,20);
8
9 -- Lossless-join check: rejoin the three projections
10 SELECT ma.MovieID, ma.ActorID, mp.PlatformID
11 FROM MovieActor ma
12 JOIN MoviePlatform mp ON ma.MovieID=mp.MovieID
13 JOIN ActorPlatform ap ON ma.ActorID=ap.ActorID AND mp.PlatformID=ap.PlatformID;

```

MovieID	ActorID	PlatformID
1	1	10
1	1	20
1	2	10
1	2	20